

Algorithmes et Pensée Computationnelle

Programmation orientée objet - Exercices basiques

Le but de cette séance est de se familiariser avec un paradigme de programmation couramment utilisé : la Programmation Orientée Objet (POO). Ce paradigme consiste en la définition et en l'interaction avec des briques logicielles appelées *Objets*. Dans les exercices suivants, nous manipulerons des objets, aborderons les notions de classe, méthodes, attributs et encapsulation. Au terme de cette séance, vous serez en mesure d'écrire des programmes mieux structurés.

Les langages de programmation qui seront utilisés pour cette série d'exercices sont Java et Python.

Le temps indiqué (🕒) est à titre strictement indicatif.

1 Itération en Java

Question 1: (🕒 5 minutes)

Qu'affiche le programme Java suivant (on n'écrit pas la classe ou la méthode `main` par souci de concision) :

```
1 String result = "";
2 String x = "5";
3 int j = 0;
4 for (j=Integer.parseInt(x); j !=0; j = j/2){
5     result = String.valueOf(j%2) + result;
6 }
7 System.out.println(result + j);
```

Choisissez parmi les éléments suivants :

- 5210
- 1110
- 1010
- 1011
- java: bad operand types for binary operator '+' ..

>_ Solution

La bonne réponse est 1010. La boucle `for` calcule la représentation binaire de l'entier stocké dans la chaîne `x` (qui est 101) et l'affichage montre la concaténation de la variable `result` et `j`. Ici, `j` est définie hors de la portée de la boucle `for`, et cela est la raison pour laquelle on peut y accéder après la boucle `for`. À la sortie de la boucle `for`, la valeur de `j` est zéro.

2 Récursion en Java et Python (10 minutes)

Question 2: (🕒 10 minutes) `isPalindrome()` — **Python et Java** Un palindrome est une chaîne de caractères qui se lit de la même manière à l'envers. Plus précisément, soit f une fonction qui renvoie `True` si a (de longueur n caractères) est un palindrome :

$$f(a) = f(a[1 : n - 1]) \wedge (a[0] == a[n - 1]) \quad (1)$$

Ici, $a[1 : n - 1]$ est la sous chaîne de a qui comprend tous les caractères à l'exception du premier caractère (le premier caractère étant à l'indice 0) et du dernier caractère (le dernier caractère étant à l'indice $n - 1$).

💡 Conseil

Revoir la question 16 du TP3 pour voir les règles concernant la notation des bornes d'une sous chaîne de caractères.

On dit donc que a est un palindrome si le premier caractère et le dernier caractère sont égaux **et** la sous chaîne entre les deux est un palindrome.

Le cas de base (une chaîne a d'un seul caractère) est :

$$f(a) = \text{True si } n = 1$$

et le case de base d'une chaîne vide : $f("") = \text{True}$.

Ecrivez la méthode Java **isPalindrome(String arg)** et la fonction Python **isPalindrome(arg)** qui renvoient True si arg est une chaîne de caractères.

3 Création de votre première classe en Java (40 minutes)

Le but de cette première partie est de créer votre propre classe en Java. Cette classe sera une classe nommée `Dog()` représentant un chien. Elle aura plusieurs attributs et méthodes que vous implémenterez au fur et à mesure.

Question 3: (🕒 10 minutes) Création de classe et encapsulation

Commencez par créer une nouvelle classe `Dog` dans votre projet. Ensuite, créez les attributs suivants :

1. Un attribut `public String` nommé `name`
2. Un attribut `private List` nommé `tricks`
3. Un attribut `private String` nommé `race`
4. Un attribut `private int` nommé `age`
5. Un attribut `private int` nommé `mood` initialisé à 5 (correspondant à l'humeur du chien)
6. Un attribut de classe (`static`) `private int` nommé `nb_chiens`

Créez une méthode publique du même nom que la classe (`Dog`). Cette méthode est appelée le constructeur, elle va servir à initialiser les différentes instances de notre classe. Un constructeur en Java aura le même nom que la classe, et le constructeur en Python sera défini par la méthode `__init__`. Cette méthode prendra en argument les éléments suivants qui seront utilisés pour initialiser les attributs de notre instance :

1. Une chaîne de caractères `name`,
2. Une liste `tricks`,
3. Une chaîne de caractères `race`,
4. Un entier `age`.

Pour finir, cette méthode doit incrémenter l'attribut de classe `nb_chiens` qui va garder en mémoire le nombre d'instances créées.

💡 Conseil

Pour revoir les notions de base du langage Java, n'hésitez pas à consulter le guide de démarrage en Java sur Moodle : <https://moodle.unil.ch/mod/folder/view.php?id=1761756>.

Pour cet exercice, n'oubliez pas de préciser si vos attributs sont `public` ou `private`.

Le mot `static` correspond à un élément de classe (attribut ou méthode), cet élément pourra ensuite être appelé via la classe directement.

Pour attribuer des valeurs à vos attributs d'instance, utilisez le mot-clé `this.attribut`.

Pour accéder aux attributs de classe, utilisez `nom_classe.nom_attribut`.

>_ Solution

```
1 public class Dog {
2     public String name;
3     private List tricks;
4     private String race;
5     private int age;
6     private int mood = 5;
7     private static int nb_chiens = 0;
8
9     public Dog(String name, List tricks, String race, int age) {
10         this.name = name;
11         this.race = race;
12         this.tricks = tricks;
13         this.age = age;
14         nb_chiens++;
15     }
16 }
```

Question 4: (🕒 10 minutes) Getters et setters

Il faut maintenant créer des méthodes de type `getter` et `setter` afin d'interagir avec les attributs `private` des instances de la classe. Les `getters` renverront les attributs souhaités tandis que les `setters` les modifieront.

Les `setters` sont souvent utilisés pour modifier la valeur d'attributs privés et ne renvoient rien.

💡 Conseil

Exemple de getters et de setters :

```
1 public String getName() { // Exemple de getter
2     return name;
3 }
4
5 public void setName(String name) { // Exemple de setter
6     this.name = name;
7 }
```

Vous pouvez directement accéder à des attributs publics en utilisant `nom_instance.attribut` à l'intérieur ou à l'extérieur de la classe.

Créez les méthodes suivantes :

- `getTricks()`
- `getRace()`
- `getAge()`
- `getMood()`
- `setTricks()`
- `setRace()`
- `setAge()`
- `setMood()`

Créez également une méthode de classe permettant de retourner le nombre de `Dog` instanciés (un `getter`).

Conseil

IntelliJ vous permet de générer automatiquement certaines méthodes telles que les getters et setters. Vous pouvez consulter le lien suivant pour plus d'informations : <https://www.jetbrains.com/help/idea/generating-code.html#generate-delegation-methods>.

Toutefois, pour cet exercice, nous vous encourageons à le faire manuellement.

>_ Solution

```
1    public List getTricks() {
2        return tricks;
3    }
4
5    public int getAge() {
6        return age;
7    }
8
9    public int getMood() {
10       return mood;
11    }
12
13    public String getRace() {
14        return race;
15    }
16
17    public static int getNb_chiens() {
18        return nb_chiens;
19    }
20
21    public void setTricks(List tricks){
22        this.tricks=tricks;
23    }
24
25    public void setAge(int age) {
26        this.age = age;
27    }
28
29    public void setMood(int mood) {
30        this.mood = mood;
31    }
32
33    public void setRace(String race) {
34        this.race = race;
35    }
```

Question 5: 5 minutes) Manipulation d'attributs - Listes

Créez une méthode publique nommée `addTrick(String trick)` qui prend en entrée une chaîne de caractères et l'ajoute à la liste `tricks`.

Conseil

La liste `tricks` est une liste comme les autres. Si vous voulez la modifier, vous aurez besoin de passer par une `LinkedList` temporaire.

>_ Solution

```
1 public void add_trick(String trick) {
2     LinkedList temp = new LinkedList(this.tricks);
3     temp.add(trick);
4     this.tricks = temp;
5 }
```

Question 6: (🕒 5 minutes) Manipulation d'attributs - setter

Créez deux méthodes permettant de modifier l'attribut mood de l'objet Dog. La méthode leash() décrémentera mood de 1 et eat() l'incrémentera de 3.

>_ Solution

```
1 public void eat() {
2     this.mood = mood + 3;
3 }
4
5 public void leash() {
6     this.mood --;
7 }
```

Question 7: (🕒 5 minutes) Manipulation d'attributs d'une autre instance

Créez une méthode nommée getOldest (Dog other) qui prend comme argument un élément de type Dog, puis retourne le nom et l'âge du chien le plus âgé sous le format suivant : "nomChien est le chien le plus âgé avec ageChien ans".

💡 Conseil

L'élément Dog que vous passez en argument est un objet de type Dog, vous pouvez donc lui appliquer les méthodes que vous avez créé tout à l'heure. Faites attention à la façon d'accéder aux différents attributs de votre deuxième chien (pour rappel, les attributs privés ne sont accessibles qu'à travers des getters que vous aurez préalablement définis).

>_ Solution

```
1 public String getOldest(Dog other) {
2     if (other.getAge() < this.getAge()) {
3         return this.name + " est le chien le plus âgé avec " +
4             this.age + " ans";
5     }
6     else{
7         return other.name + " est le chien le plus âgé avec " +
8             other.getAge() + " ans";
9     }
10 }
```

Question 8: (🕒 5 minutes) Redéfinition de méthodes

Créez une méthode toString() de type public qui retourne une chaîne de caractères contenant toutes les informations d'une instance de Dog. Ainsi, dans votre main, en faisant System.out.println(...)

sur une instance de Dog, vous obtiendrez un texte sous le format suivant : “nomChien a ageChien ans, est un raceChien et a une humeur de moodChien. Il sait faire les tours suivants : tricksChien”.

💡 Conseil

La méthode `toString()` permet d’obtenir une représentation textuelle d’un objet. Elle est définie par défaut pour tout objet. Pour la redéfinir, on utilise le mot-clé `@Override` avant la méthode.

>_ Solution

```
1 @Override
2 public String toString(){return this.name + " a " + this.age +
    " ans, est un " + this.race +
3     " et a une humeur de " + this.mood + ". Il sait
    faire les tours suivants : " + this.tricks;}
```

Pour contrôler que vos méthodes et attributs ont été implémentés correctement, vous pouvez essayer le code suivant à l’intérieur de votre méthode `main` :

```
1 public class Main {
2     public static void main(String[] args) {
3         Dog lola = new Dog("Lola",List.of("rollover"),"Bouvier",10);
4         Dog tobi = new Dog("Tobi",List.of("rollover","do a
    barrel"),"Doggo",17);
5         System.out.println(lola.getAge());
6         System.out.println(lola.getMood());
7         System.out.println(lola.getRace());
8         System.out.println(lola.name);
9         System.out.println(lola.getTricks());
10        lola.setAge(13);
11        lola.setMood(8);
12        lola.setRace("Bouvier");
13        lola.name = "lola";
14        lola.setTricks(List.of("rollover","do a barrel"));
15        lola.eat();
16        lola.leash();
17        lola.addTrick("sit");
18        System.out.println(Dog.getNbChiens());
19        System.out.println(lola.getOldest(tobi));
20        System.out.println(lola);
21    }
22 }
```

Vous devriez obtenir ce résultat :

```
1 10
2 5
3 Bouvier
4 Loola
5 [rollover]
6 2
7 Tobi est le chien le plus agé avec 17 ans
8 Lola a 13 ans, est un Bouvier et a une humeur de 10. Il sait faire les
    tours suivants : [rollover, do a barrel, sit]
```

4 Notions de POO en Python (15 minutes)

Dans cette section, nous créerons pas-à-pas une classe `Point` contenant des attributs et des méthodes utiles. Dans votre IDE, créez un nouveau projet Python (Fichier > Nouveau > Projet). Dans un dossier de votre choix, créez un fichier `question7.py`.

Question 9: (🕒 15 minutes) Classe `Point`

- Créez une classe `Point` et un constructeur par défaut contenant deux paramètres (`x` et `y`).

💡 Conseil

Pour rappel, un constructeur est une fonction `__init__()` que vous redéfinirez dans votre classe.

- Définissez deux attributs privés pour votre classe `Point`. Ces attributs seront les coordonnées `x` et `y` de vos points. Par défaut, assignez leur les valeurs données dans le constructeur.

💡 Conseil

À l'intérieur d'une classe, utilisez le mot-clé `self` pour accéder aux méthodes et attributs de l'instance que vous manipulez. En Python, pour spécifier qu'un attribut est privé, rajouter un double underscore au nom de l'attribut (Exemple : `__score=0`)

- Définir des getters et setters.

💡 Conseil

En Python, le mot-clé `self` est l'équivalent de `this` utilisé en Java.

- Définissez une méthode `distance` qui prend en entrée l'instance du `Point` (`self`) et un autre `Point` `p2`. Cette méthode `distance` retournera la distance euclidienne entre le point `self` et `p2`.

💡 Conseil

Pour rappel, la distance euclidienne entre deux points est définie par la formule $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. Utilisez la fonction `sqrt()` de la librairie `math` pour calculer la racine carrée. Pensez à importer la librairie `math`.

- Définissez une méthode `milieu` qui prendra en entrée `self` et `p2` et qui retournera un objet `Point` situé entre `self` et `p2`.

💡 Conseil

Pour trouver les coordonnées d'un point $M(x_M, y_M)$ situé au milieu du segment défini par des points $A(x_A, y_A)$ et $B(x_B, y_B)$, utilisez les formules suivantes : $x_M = \frac{x_1 + x_2}{2}$ et $y_M = \frac{y_1 + y_2}{2}$

- Redéfinissez une méthode `__str__()` dans la classe `Point` qui retournera une chaîne de caractères contenant les coordonnées (x, y) d'un point. Ainsi, lorsqu'on fera un `print` d'une instance de la classe `Point`, le message qui s'affichera sera le suivant : *Les coordonnées du Point sont : x = "remplacez par la valeur de x" et y = "remplacez par la valeur de y"*

>_ Solution

```
1 import math
2
3 class Point:
4     def __init__(self, x, y):
5         self.__x = x
6         self.__y = y
7
8     def get_x(self):
9         return self.__x
10
11    def get_y(self):
12        return self.__y
13
14    def set_x(self, x):
15        self.__x = x
16
17    def set_y(self, y):
18        self.__y = y
19
20    def distance(self, p2):
21        return math.sqrt((self.__x - p2.get_x())**2 + (self.__y
22        - p2.get_y())**2)
23
24    def milieu(self, p2):
25        x_M = (self.__x + p2.get_x()) / 2
26        y_M = (self.__y + p2.get_y()) / 2
27        M = Point(x_M, y_M)
28        return M
29
30    def __str__(self):
31        return "Les coordonnées du point sont:
32        x="+str(self.get_x())+", y="+str(self.get_y())
33
34    if __name__ == '__main__':
35        p = Point(3, 2)
36        p2 = Point(5,4)
37        print(str(p.distance(p2)))
38        print(str(p.milieu(p2)))
```


5 Notions d'héritage - Java (30 minutes)

Le but de cette partie est de mettre en pratique les notions liées à l'héritage. Nous allons créer une classe `Livre()` qui représentera notre classe-mère. Nous allons également créer deux classes filles, `Livre_Audio()` et `Livre_Illustre()`. Les classes filles hériteront des attributs et méthodes de la classe-mère.

Question 10: (🕒 20 minutes) Création des différentes classes
Créez la classe-mère `Livre` avec les caractéristiques suivantes :

- un attribut privé `String` nommé `titre`,
- un attribut privé `String` nommé `auteur`,
- un attribut privé `int` nommé `annee`,
- un attribut privé `int` nommé `note` (initialisé à `-1`),
- le constructeur de la classe qui prendra les trois premiers arguments cités ci-dessus,
- une méthode `setNote()` qui permet de définir l'attribut `note`,
- une méthode `getNote()` qui permet de retourner l'attribut `note`,
- une méthode `toString()` qui retournera le titre, l'auteur, l'année et la note d'un ouvrage `note` (réécrire cette méthode permettra d'afficher un objet `Livre` en utilisant `System.out.println()`).

Attention, si la `note` n'a pas été modifiée et qu'elle vaut toujours `-1`, affichez "Note : pas encore attribuée" au lieu de "Note : `note`" via la méthode `toString()`.

Créez les classes filles avec les caractéristiques suivantes :

```
class Livre_Audio extends Livre
```

- un attribut privé `String` nommé `narrateur`

```
class Livre_Illustre extends Livre
```

- un attribut privé `String` nommé `illustrateur`

Voici le squelette du programme à compléter :

```
1      class Livre {  
2      }  
3      class Livre_Audio extends Livre {  
4      }  
5      class Livre_Illustre extends Livre {  
6      }
```

💡 Conseil

- En Java, lors de la déclaration d'une classe, le mot clé `extends` permet d'indiquer qu'il s'agit d'une classe fille de la classe indiquée. Le mot clé `super` permet à la sous-classe d'hériter d'éléments de la classe-mère. `super` peut être utilisé dans le constructeur de la classe-fille selon l'exemple suivant : `super(attribut_mère_1, attribut_mère_2, attribut_mère_3, etc.)`; . Ainsi, il n'est pas nécessaire de redéfinir tous les attributs d'une classe fille !
- L'instruction `super` doit toujours être la première instruction dans le constructeur d'une classe-fille.
- Vous pouvez vous servir de `'\n'` dans une chaîne de caractères pour effectuer un retour à la ligne lors de l'affichage d'une chaîne de caractères.

>_ Solution

```
1 public class Livre {
2
3     private String titre;
4     private String auteur;
5     private int annee;
6     private int note = -1;
7
8     public Livre(String titre, String auteur, int annee){
9         System.out.println("Création d'un livre");
10        this.titre = titre;
11        this.auteur = auteur;
12        this.annee = annee;
13    }
14
15    public int getNote(){
16        return this.note;
17    }
18
19    public void setNote(int note) {
20        this.note = note;
21    }
22
23    public String toString() {
24        if (note == -1){
25            return "A propos du livre \n----- \nTitre
26            : " +titre+ "\nAuteur : "+auteur+ "\nAnnée : "+annee+
27            "\nNote : non attribuée";
28        }
29        else{
30            return "A propos du livre \n----- \nTitre
31            : " +titre+ "\nAuteur : "+auteur+ "\nAnnée : "+annee+
32            "\nNote : "+note;
33        }
34    }
35
36    class Livre_Audio extends Livre {
37        private String narrateur;
38
39        public Livre_Audio(String titre, String auteur, int annee,
40        String narrateur){
41            super(titre, auteur, annee);
42            System.out.println("Création d'un livre audio");
43            this.narrateur = narrateur;
44        }
45    }
46
47    class Livre_Illustre extends Livre {
48        private String illustateur;
49
50        public Livre_Illustre(String titre, String auteur, int
51        annee, String illustateur) {
52            super(titre, auteur, annee);
53            System.out.println("Création d'un livre illustré");
54            this.illustateur = illustateur;
55        }
56    }
```

Question 11: (🕒 10 minutes) Méthode et héritage

Maintenant que vous avez créé la classe-mère et les classes filles correspondantes, vous pouvez créer un objet `Livre` à l'aide du constructeur de la classe `Livre_Audio` (et des arguments donnés lors de la création de l'objet). En instanciant l'objet, vous pourriez utiliser les valeurs suivantes :

titre : "Hamlet", auteur : "Shakespeare", année : "1609" et le narrateur "William".

Une fois l'objet créé, attribuez-lui une note à l'aide de la méthode `setNote()` définie précédemment.

Finalement, utilisez la méthode `System.out.println()` pour afficher les informations du livre.

Redéfinir la méthode `toString()` de la classe `Livre_Audio` afin que la valeur de l'attribut `narrateur` soit affichée. Faites pareil avec la classe `Livre_Illustre` et son attribut `Illustrateur`

💡 Conseil

Attention, vous devez créer un objet `Livre` et non `Livre_Audio`. Le mot-clé `super` peut être utilisé dans la redéfinition d'une méthode selon l'exemple suivant : `super.nom_de_la_methode();`. Le mot clé `super` représente la classe parent, tout comme le mot clé `this` fait référence à l'instance avec laquelle la méthode est appelée. L'instruction `super` doit toujours être la première instruction dans le redéfinition d'une méthode dans une classe fille.

>_ Solution

```
1 class Livre_Audio extends Livre {
2
3     private String narrateur;
4
5     public Livre_Audio(String titre, String auteur, int annee,
6         String narrateur){
7         super(titre, auteur, annee);
8         System.out.println("Création d'un livre audio");
9         this.narrateur = narrateur;
10    }
11
12    // redéfinition de la fonction toString dans la classe
13    // fille Livre_Audio
14    public String toString() {
15        return super.toString() + "\nNarrateur: " +
16        narrateur+"\n"; //Ajoute narrateur à la chaîne de caractère
17        // créée par la classe mère (super)
18    }
19 }
20
21 class Livre_Illustre extends Livre {
22
23     private String illustrateur;
24
25     public Livre_Illustre(String titre, String auteur, int
26         annee, String illustrateur) {
27         super(titre, auteur, annee);
28         System.out.println("Création d'un livre illustré");
29         this.illustrateur = illustrateur;
30    }
31
32    public String toString() {
33        return super.toString() + "\nIllustrateur: " +
34        illustrateur + "\n"; //Ajoute illustrateur à la chaîne de
35        // caractère créée par la classe mère (super)
36    }
37 }
38
39 public class Main {
40
41     public static void main(String[] args) {
42         Livre Livrel = new Livre_Audio("Hamlet", "Shakespeare",
43         1609, "William");
44         Livrel.setNote(5);
45         System.out.println(Livrel);
46     }
47 }
48 }
```

Lorsque toutes les étapes auront été effectuées, placez le code suivant dans votre `main` et exécutez votre programme :

```
1 public class Main {
2
3     public static void main(String[] args) {
4         Livre Livrel = new Livre_Audio("Hamlet", "Shakespeare",
5         1609, "William");
6         Livrel.setNote(5);
7         System.out.println(Livrel);
8         Livre Livre2 = new Livre("Les Misérables", "Hugo", 1862);
9         System.out.println(Livre2);
10    }
```

Vous devriez obtenir :

```
1 Création d'un livre
2 Création d'un livre audio
3 Création d'un livre
4 A propos du livre
5 -----
6 Titre : Hamlet
7 Auteur : Shakespeare
8 Année : 1609
9 Note : 5
10 Narrateur: William
11 A propos du livre
12 -----
13 Titre : Les Misérables
14 Auteur : Hugo
15 Année : 1862
16 Note : non attribuée
17 Process finished with exit code 0
```